

Problem Set 2 Report: Working with Prime Numbers

Overview

This assignment focused on implementing generate-and-test algorithms in Python to work with prime numbers. Two separate programs were created using Python and Visual Studio Code. The first program identified the 450th prime number through systematic iteration and prime number testing. The second program evaluated the relationship between prime numbers and logarithmic summation to observe numerical convergence behavior.

The assignment demonstrated several programming concepts discussed throughout the course, including iteration, modularity, conditional logic, algorithmic efficiency, and mathematical computation.

Problem One: Finding the 450th Prime Number

The first program used a generate-and-test approach to identify the 450th prime number. Instead of testing every positive integer, the program optimized performance by skipping even numbers greater than two because all prime numbers other than two are odd. A custom function named `is_prime()` was created to determine whether a candidate number was prime.

The program also improved efficiency by only testing divisors up to the square root of the candidate number. If no divisors were found within that range, the number was considered prime.

During execution, the program printed progress updates after every 50 prime numbers discovered. The output demonstrated that the algorithm successfully identified the 450th prime number as 3181.

Output included:

```
P5 C:\Users\alanp\AppData\Local\Programs\Microsoft VS Code> & C:\Users\alanp\AppData\Local\Microsoft\WindowsApps\python3.13.exe "///TRUENAS/ShadowRealm/Al
n's Prometheus Backup/Education/US College/University of the Cumberland/MSDS-532/assignment_2_1.py"
50 prime numbers found so far.
100 prime numbers found so far.
150 prime numbers found so far.
200 prime numbers found so far.
250 prime numbers found so far.
300 prime numbers found so far.
350 prime numbers found so far.
400 prime numbers found so far.
450 prime numbers found so far.
The program was looking for the 450th prime number.
The 450th prime number is 3181.
```

This problem demonstrated how systematic iteration and conditional testing can efficiently solve mathematical problems using programming logic.

Problem Two: Sum of Logarithms of Prime Numbers

The second program evaluated the mathematical observation that the sum of the logarithms of prime numbers between 2 and n converges toward the value of n . The program reused the primetesting function from Problem One and calculated the sum of natural logarithms for all prime numbers within several ranges of n .

The following values were evaluated:

n	Sum of Prime Logarithms	Ratio (sum / n)
50	40.9602	0.8192
500	474.5544	0.9491
5000	4911.6954	0.9823
50000	49732.0184	0.9946

The results demonstrated that as n increased, the ratio between the logarithmic sum and n moved closer to 1. This supports the convergence behavior described in the assignment instructions.

```
PS C:\Users\alanp\AppData\Local\Programs\Microsoft VS Code> & C:\Users\alanp\AppData\Local\Microsoft\WindowsApps\python3.13.exe "/:/TRUENAS/ShadowRealm/Ala
n's Prometheus Backup/Education/US College/University of the Cumberland/MSDS-532/assignment_2_2.py"
For n = 50:
Sum of logarithms of prime numbers = 40.960219431352954
Ratio of sum to n = 0.819204388627059

For n = 500:
Sum of logarithms of prime numbers = 474.5544441546942
Ratio of sum to n = 0.9491088883093883

For n = 5000:
Sum of logarithms of prime numbers = 4911.695351706861
Ratio of sum to n = 0.9823390703413722

For n = 50000:
Sum of logarithms of prime numbers = 49732.01840295681
Ratio of sum to n = 0.9946403680591362
```

The program also demonstrated modular programming practices by separating prime-testing logic into reusable functions. Additionally, the assignment reinforced the importance of computational efficiency because larger values of n significantly increase the number of iterations and calculations required.

Conclusion

Overall, this assignment demonstrated the practical use of generate-and-test algorithms, iteration, mathematical computation, and modular programming in Python. The first problem successfully identified the 450th prime number using optimized prime-testing logic, while the second problem demonstrated the convergence relationship between prime logarithmic sums and increasing values of n . These exercises reinforced important programming concepts related to efficiency, algorithm design, and reusable code structure.